```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!git clone https://github.com/indobenchmark/indonlu.git
%cd indonlu
```

```
Cloning into 'indonlu'...
remote: Enumerating objects: 509, done.
remote: Counting objects: 100% (193/193), done.
remote: Compressing objects: 100% (83/83), done.
remote: Total 509 (delta 119), reused 139 (delta 110), pack-reused 316 (from 1)
Receiving objects: 100% (509/509), 9.46 MiB | 14.99 MiB/s, done.
Resolving deltas: 100% (239/239), done.
/content/indonlu
```

```
!pip install google-play-scraper
!pip install pandas
!pip install numpy
!pip install torch
!pip install scikit-learn
!pip install Sastrawi
!pip install nltk
!pip install transformers
```

```
Collecting google-play-scraper
  Downloading google_play_scraper-1.2.7-py3-none-any.whl.metadata (50 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 50.2/50.2 kB 3.0 MB/s eta 0:00:00
Downloading google_play_scraper-1.2.7-py3-none-any.whl (28 kB)
Installing collected packages: google-play-scraper
Successfully installed google-play-scraper-1.2.7
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (2.2
Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.11/dist-packag
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/di
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-package
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packa
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (f
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (2.0.
Requirement already satisfied: torch in /usr/local/lib/python3.11/dist-packages (2.6.
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (f
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.11
Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (f
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-packages (frc
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (frc
Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metac
Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.met
Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metac
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch)
```

```
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl.metadata (
Collecting nvidia-cublas-cu12==12.4.5.8 (from torch)
  Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl.metadata
Collecting nvidia-cufft-cu12==11.2.1.3 (from torch)
  Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl.metadata (
Collecting nvidia-curand-cu12==10.3.5.147 (from torch)
  Downloading nvidia_curand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.whl.metadat
Collecting nvidia-cusolver-cu12==11.6.1.9 (from torch)
  Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.whl.metadat
Collecting nvidia-cusparse-cu12==12.3.1.170 (from torch)
  Downloading nvidia_cusparse_cu12-12.3.1.170-py3-none-manylinux2014_x86_64.whl.metac
Requirement already satisfied: nvidia-cusparselt-cu12==0.6.2 in /usr/local/lib/pythor
Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in /usr/local/lib/python3.11/
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in /usr/local/lib/python3.1
Collecting nvidia-nvjitlink-cu12==12.4.127 (from torch)
  Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metada
Requirement already satisfied: triton==3.2.0 in /usr/local/lib/python3.11/dist-packag
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-packag
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/dist-p
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-pack
Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl (363.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 363.4/363.4 MB 4.2 MB/s eta 0:00:00
Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (13.8 M
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13.8/13.8 MB 72.4 MB/s eta 0:00:00
Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (24.6 M
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.6/24.6 MB 27.9 MB/s eta 0:00:00
Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (883
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 883.7/883.7 kB 60.2 MB/s eta 0:00:00
Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (664.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 664.8/664.8 MB 2.0 MB/s eta 0:00:00
Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl (211.5 MB)
```

```
import pandas as pd
import numpy as np
import re
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from collections import defaultdict

# NLP
import nltk
nltk.download('punkt')
# from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
from nltk.probability import FreqDist
from nltk.tokenize import word_tokenize

# Viz
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib as mpl
from wordcloud import WordCloud
```

```
#Model IndoBERT
import random
import torch
import torch.nn.functional as F
from torch import optim
from tqdm import tqdm

from transformers import BertForSequenceClassification, BertConfig, BertTokenizer
from indonlu.utils.data_utils import DocumentSentimentDataset, DocumentSentimentDataLoader
from indonlu.utils.forward_fn import forward_sequence_classification
from indonlu.utils.metrics import document_sentiment_metrics_fn
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

```
import multiprocessing
print(multiprocessing.cpu_count())
```

```
2
```

Finetuning IndoBERT

```
import random
import numpy as np
import torch

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)

def count_param(module, trainable=False):
    if trainable:
        return sum(p.numel() for p in module.parameters() if p.requires_grad)
    else:
        return sum(p.numel() for p in module.parameters())

def get_lr(optimizer):
    for param_group in optimizer.param_groups:
        return param_group['lr']

def metrics_to_string(metric_dict):
    string_list = []
    for key, value in metric_dict.items():
        string_list.append('{}:{:.2f}'.format(key, value))
    return ' '.join(string_list)
```

```
# Set random seed
set_seed(27)
```

Load Model

```
from indonlu.utils.data_utils import DocumentSentimentDataset, DocumentSentimentDataLoader
from indonlu.utils.forward_fn import forward_sequence_classification
from indonlu.utils.metrics import document_sentiment_metrics_fn


from transformers import BertTokenizer, BertConfig, BertForSequenceClassification

# Load Tokenizer and Config
tokenizer = BertTokenizer.from_pretrained('indobenchmark/indobert-base-p1')
config = BertConfig.from_pretrained('indobenchmark/indobert-base-p1')
config.num_labels = DocumentSentimentDataset.NUM_LABELS

# Instantiate model
model = BertForSequenceClassification.from_pretrained('indobenchmark/indobert-base-p1', conf
```

```
Some weights of BertForSequenceClassification were not initialized from the model checkp
You should probably TRAIN this model on a down-stream task to be able to use it for prec
```

```
# Struktur model
model
```

```
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(50000, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSdpaSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
```

```
                (dropout): Dropout(p=0.1, inplace=False)
              )
            )
            (intermediate): BertIntermediate(
              (dense): Linear(in_features=768, out_features=3072, bias=True)
              (intermediate_act_fn): GELUActivation()
            )
            (output): BertOutput(
              (dense): Linear(in_features=3072, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
        )
      )
      (pooler): BertPooler(
        (dense): Linear(in_features=768, out_features=768, bias=True)
        (activation): Tanh()
      )
    )
    (dropout): Dropout(p=0.1, inplace=False)
    (classifier): Linear(in_features=768, out_features=3, bias=True)
  )
```

Prepare Dataset

```
train_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_train_70.tsv'
valid_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_validation_15.
test_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_test_15.tsv'


train_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_train_80.tsv'
valid_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_validation_10.
test_dataset_path = '/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_ds_test_10.tsv'


# fungsi dataset loader dari utils IndoNLU
train_dataset = DocumentSentimentDataset(train_dataset_path, tokenizer, lowercase=True)
valid_dataset = DocumentSentimentDataset(valid_dataset_path, tokenizer, lowercase=True)
test_dataset = DocumentSentimentDataset(test_dataset_path, tokenizer, lowercase=True)

train_loader = DocumentSentimentDataLoader(dataset=train_dataset, max_seq_len=512, batch_siz
valid_loader = DocumentSentimentDataLoader(dataset=valid_dataset, max_seq_len=512, batch_siz
test_loader = DocumentSentimentDataLoader(dataset=test_dataset, max_seq_len=512, batch_size=


w2i, i2w = DocumentSentimentDataset.LABEL2INDEX, DocumentSentimentDataset.INDEX2LABEL
print(w2i) #word to index
print(i2w) #index to word
```

```
{'positive': 0, 'neutral': 1, 'negative': 2}
{0: 'positive', 1: 'neutral', 2: 'negative'}
```

Uji coba pre-trained model

```
import torch.nn.functional as F


text = 'tidak bisa transfer antar bank'
subwords = tokenizer.encode(text)
subwords = torch.LongTensor(subwords).view(1, -1).to(model.device)

logits = model(subwords)[0]
label = torch.topk(logits, k=1, dim=-1)[1].squeeze().item()

print(f'Text: {text} | Label : {i2w[label]} ({F.softmax(logits, dim=-1).squeeze()[label] * 1
```

```
Text: tidak bisa transfer antar bank | Label : neutral (55.560%)
```

Fine Tuning & Prediksi Evaluation

```
# Tentukan optimizer
optimizer = optim.Adam(model.parameters(), lr=3e-5)
model = model.cuda()


# Train
n_epochs = 10
history = defaultdict(list)

for epoch in range(n_epochs):
    model.train()
    torch.set_grad_enabled(True)

    total_train_loss = 0
    list_hyp_train, list_label = [], []

    train_pbar = tqdm(train_loader, leave=True, total=len(train_loader))
    for i, batch_data in enumerate(train_pbar):
        loss, batch_hyp, batch_label = forward_sequence_classification(model, batch_dat

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        tr_loss = loss.item()
        total_train_loss = total_train_loss + tr_loss
```

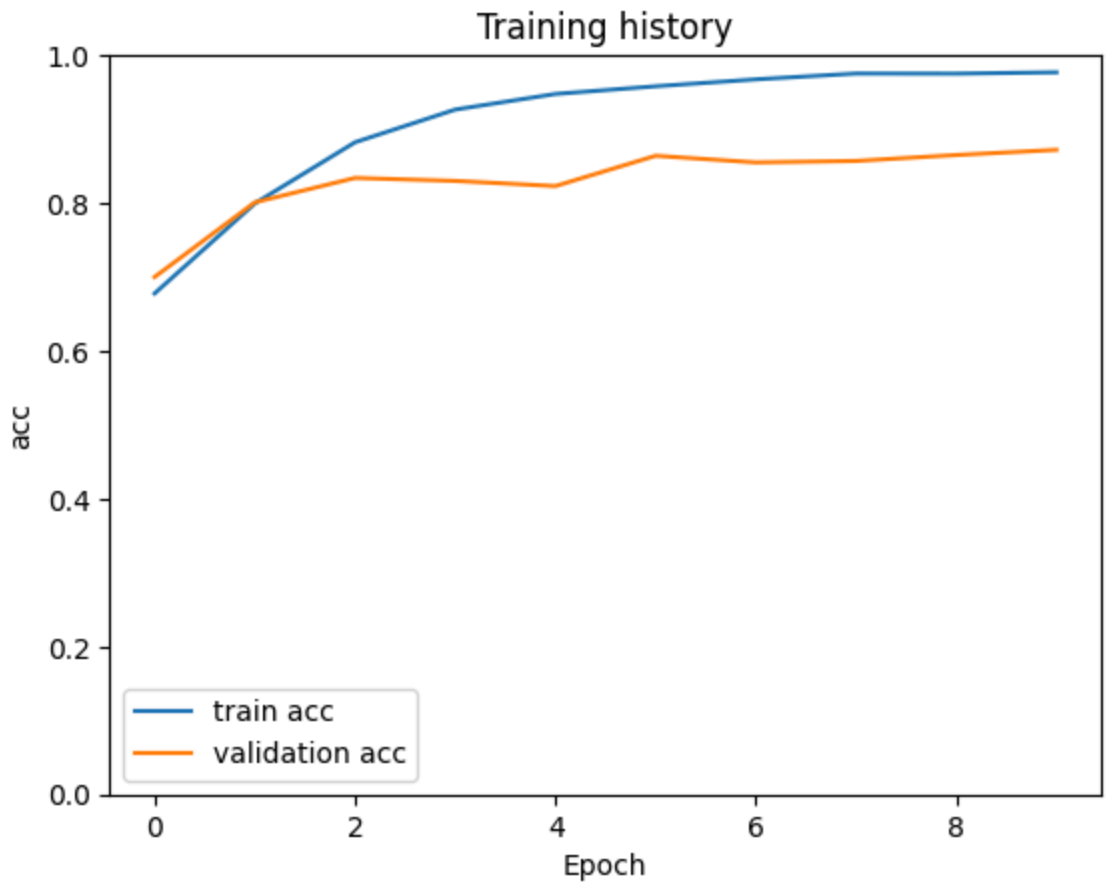
```
            total_train_loss += tr_loss

            list_hyp_train += batch_hyp
            list_label += batch_label

            train_pbar.set_description(f"(Epoch {epoch+1}) TRAINING...")

        train_metrics = document_sentiment_metrics_fn(list_hyp_train, list_label)

        print(f"\nEpoch {epoch+1} - Training Accuracy: {train_metrics['ACC']:.4f}")

        history['train_acc'].append(train_metrics['ACC'])

        # Validation
        model.eval()
        torch.set_grad_enabled(False)

        list_hyp_valid, list_label_valid = [], []

        valid_pbar = tqdm(valid_loader, leave=True, total=len(valid_loader))
        for i, batch_data in enumerate(valid_pbar):
            _, batch_hyp, batch_label = forward_sequence_classification(model, batch_data[:
            list_hyp_valid += batch_hyp
            list_label_valid += batch_label

            valid_pbar.set_description("Validating...")

        valid_metrics = document_sentiment_metrics_fn(list_hyp_valid, list_label_valid)

        print(f"Epoch {epoch+1} - Validation Accuracy: {valid_metrics['ACC']:.4f}")
        print("================================================\n")

        history['val_acc'].append(valid_metrics['ACC'])
```

```
(Epoch 3) TRAINING...: 100%|██████████| 500/500 [03:08<00:00,  2.65it/s]

Epoch 3 - Training Accuracy: 0.8822
Validating...: 100%|██████████| 63/63 [00:07<00:00,  7.89it/s]
```

```
Epoch 5 - Validation Accuracy: 0.8230
==============================================

(Epoch 6) TRAINING...: 100%|██████████| 500/500 [03:09<00:00,  2.65it/s]

Epoch 6 - Training Accuracy: 0.9580
Validating...: 100%|██████████| 63/63 [00:07<00:00,  7.89it/s]
Epoch 6 - Validation Accuracy: 0.8640
==============================================

(Epoch 7) TRAINING...: 100%|██████████| 500/500 [03:08<00:00,  2.65it/s]

Epoch 7 - Training Accuracy: 0.9674
Validating...: 100%|██████████| 63/63 [00:07<00:00,  7.93it/s]
Epoch 7 - Validation Accuracy: 0.8550
==============================================

(Epoch 8) TRAINING...: 100%|██████████| 500/500 [03:08<00:00,  2.65it/s]

Epoch 8 - Training Accuracy: 0.9752
Validating...: 100%|██████████| 63/63 [00:07<00:00,  7.94it/s]
Epoch 8 - Validation Accuracy: 0.8570
==============================================

(Epoch 9) TRAINING...: 100%|██████████| 500/500 [03:08<00:00,  2.65it/s]

Epoch 9 - Training Accuracy: 0.9751
Validating...: 100%|██████████| 63/63 [00:07<00:00,  7.91it/s]
Epoch 9 - Validation Accuracy: 0.8650
==============================================

(Epoch 10) TRAINING...: 100%|██████████| 500/500 [03:09<00:00,  2.64it/s]

Epoch 10 - Training Accuracy: 0.9769
Validating...: 100%|██████████| 63/63 [00:07<00:00,  7.91it/s]Epoch 10 - Validation A
==============================================
```

Learning Curve

```
plt.plot(history['train_acc'], label='train acc')
plt.plot(history['val_acc'], label='validation acc')
plt.title('Training history')
plt.ylabel('acc')
plt.xlabel('Epoch')
plt.legend()
plt.ylim([0, 1]);
```




```
# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_70_30_16_3e5.
```

```
# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_70_30_16_3e6.
```

```
# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_70_30_32_3e5.
```

```
# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_70_30_32_3e6.
```

```
# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_80_20_16
```

```
# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_80_20_16_3e6.
```

```
# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_80_20_32_3e5.
```

```
# Simpan Hasil Prediksi Validation Set
val_df = pd.read_csv(valid_dataset_path, sep='\t', names=['review_text', 'category'])
val_df['pred'] = list_hyp_valid
val_df.head()
val_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_val_result_80_20_32_3e6.
```

Prediksi Test Set

```
# Prediksi test set
model.eval()
torch.set_grad_enabled(False)

total_loss, total_correct, total_labels = 0, 0, 0
pred, list_label = [], []

pbar = tqdm(test_loader, leave=True, total=len(test_loader))
for i, batch_data in enumerate(pbar):
    _, batch_hyp, _ = forward_sequence_classification(model, batch_data[:-1], i2w=i2w,
    pred += batch_hyp
```

```
100%|██████████| 63/63 [00:07<00:00,  8.00it/s]
```

```
# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_70_30_16_3e
```

```
# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_70_30_16_3e
```

```
# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_70_30_32_3e
```

```
# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_70_30_32_3e
```

```
# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_80_20_
```

```
# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_80_20_16_3e
```

```
# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_80_20_32_3e
```

```
# Simpan prediksi test set
test_df = pd.read_csv(test_dataset_path, sep='\t', names=['review_text', 'category'])
test_df['pred'] = pred
```

```
test_df.head()
test_df.to_csv('/content/drive/MyDrive/Colab Notebooks/SKRIPSI/wondr_test_result_80_20_32_3e
```

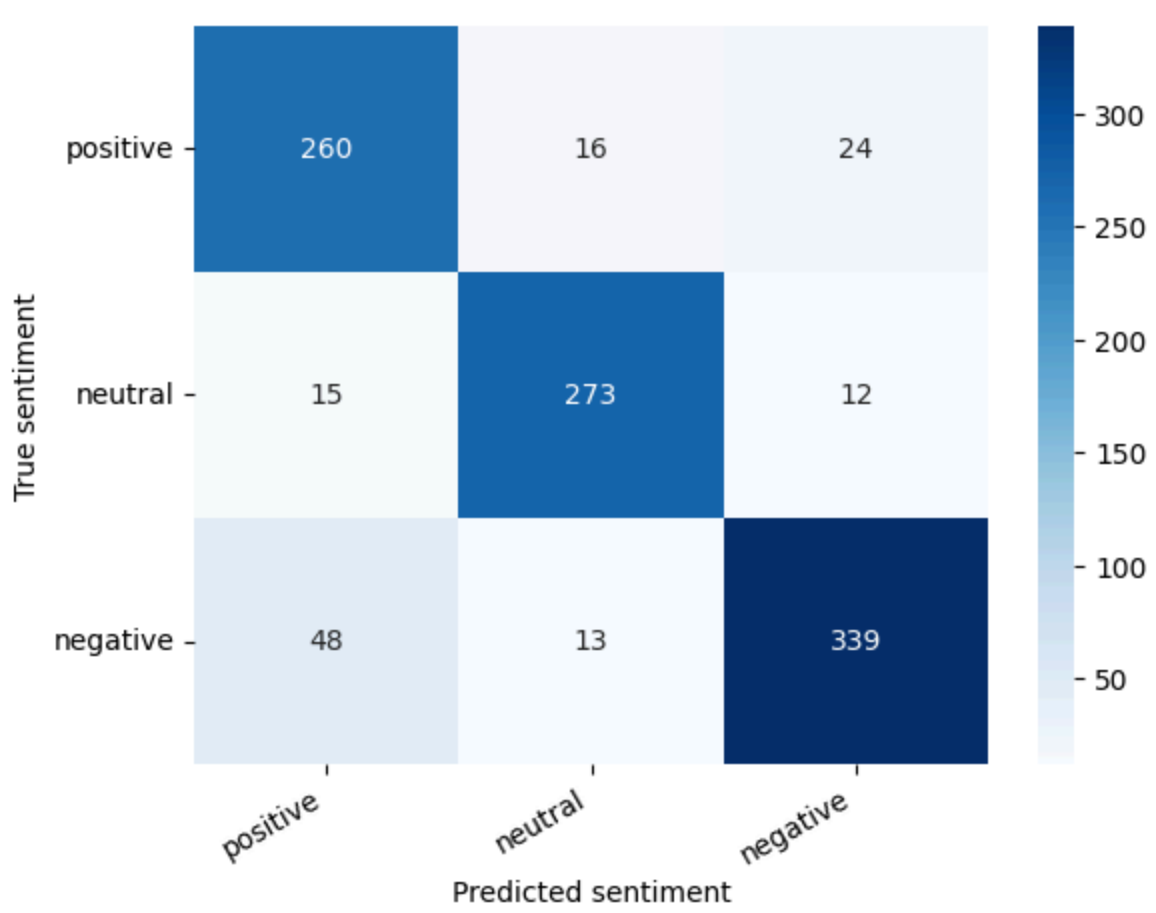
Test fine-tuned model on sample sentences

```
text = 'tidak bisa transfer antar bank'
subwords = tokenizer.encode(text)
subwords = torch.LongTensor(subwords).view(1, -1).to(model.device)

logits = model(subwords)[0]
label = torch.topk(logits, k=1, dim=-1)[1].squeeze().item()

print(f'Text: {text} | Label : {i2w[label]} ({F.softmax(logits, dim=-1).squeeze()[label] * 1
```

```
Text: tidak bisa transfer antar bank | Label : negative (99.203%)
```

Evaluasi

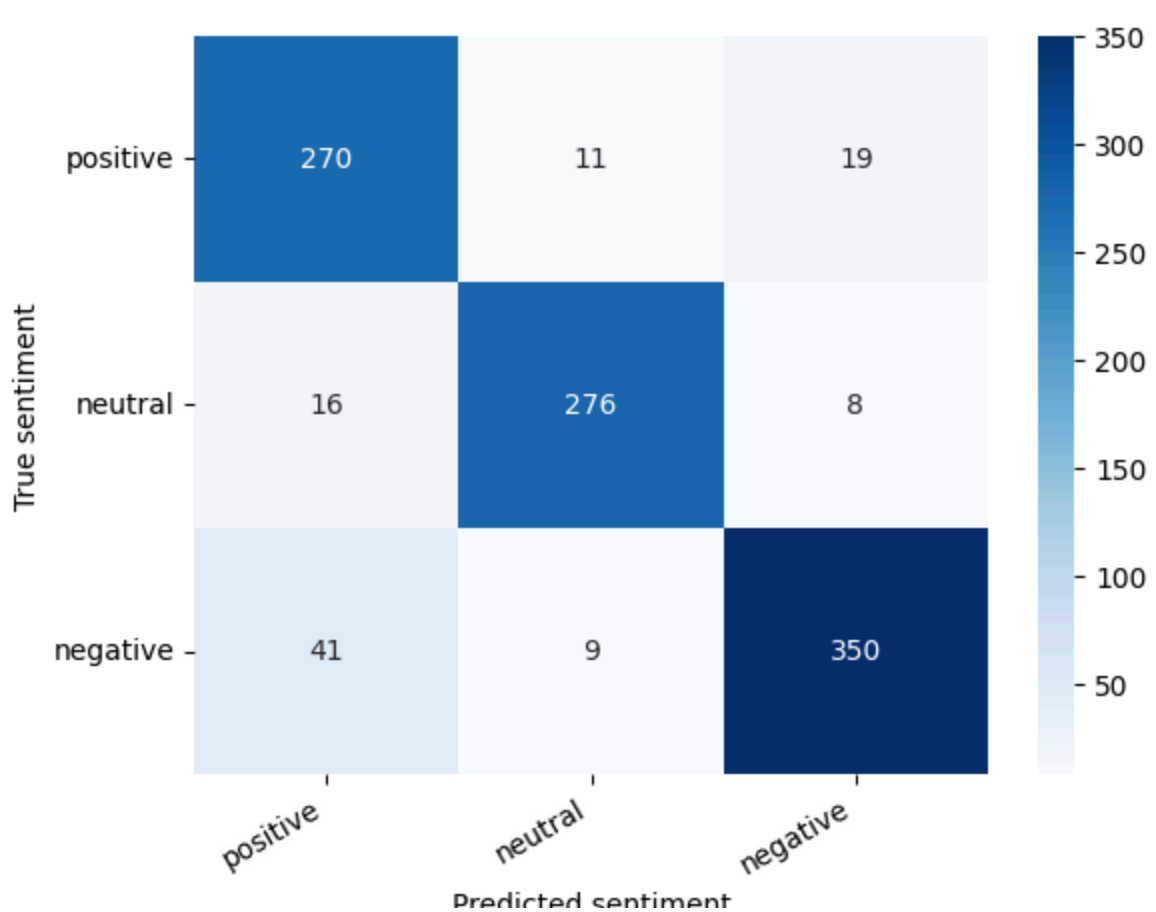
```
val_real = val_df.category
val_pred = val_df.pred

test_real = test_df.category
test_pred = test_df.pred


def show_confusion_matrix(confusion_matrix):
  hmap = sns.heatmap(confusion_matrix, annot=True, fmt="d", cmap="Blues")
  hmap.yaxis.set_ticklabels(hmap.yaxis.get_ticklabels(), rotation=0, ha='right')
  hmap.xaxis.set_ticklabels(hmap.xaxis.get_ticklabels(), rotation=30, ha='right')
  plt.ylabel('True sentiment')
  plt.xlabel('Predicted sentiment');

cm = confusion_matrix(val_real, val_pred)
df_cm = pd.DataFrame(cm, index=['positive', 'neutral', 'negative'], columns=['positive'
show_confusion_matrix(df_cm)
```




```
print(classification_report(val_real, val_pred, target_names=['positive', 'neutral', 'negati
```

```
              precision    recall  f1-score   support

    positive       0.80      0.87      0.83       300
     neutral       0.90      0.91      0.91       300
    negative       0.90      0.85      0.87       400

    accuracy                           0.87      1000
   macro avg       0.87      0.87      0.87      1000
weighted avg       0.87      0.87      0.87      1000
```

```
def show_confusion_matrix(confusion_matrix):
  hmap = sns.heatmap(confusion_matrix, annot=True, fmt="d", cmap="Blues")
  hmap.yaxis.set_ticklabels(hmap.yaxis.get_ticklabels(), rotation=0, ha='right')
  hmap.xaxis.set_ticklabels(hmap.xaxis.get_ticklabels(), rotation=30, ha='right')
  plt.ylabel('True sentiment')
  plt.xlabel('Predicted sentiment');

cm = confusion_matrix(test_real, test_pred)
df_cm = pd.DataFrame(cm, index=['positive', 'neutral', 'negative'], columns=['positive'
show_confusion_matrix(df_cm)
```




```
print(classification_report(test_real, test_pred, target_names=['positive', 'neutral',
```

```
              precision    recall  f1-score   support

    positive       0.83      0.90      0.86       300
     neutral       0.93      0.92      0.93       300
    negative       0.93      0.88      0.90       400

    accuracy                           0.90      1000
   macro avg       0.90      0.90      0.90      1000
weighted avg       0.90      0.90      0.90      1000
```